Now let's simulate reaching the rate limit. I have provided a Bash script named `load-test.sh` that you can run inside the project folder ch09/chat-validation/ to generate load:

```
%./load-test.sh
{"message":"Too many requests"}....
```

As a rule, using external rate-limiting services like Cloudflare, AWS WAF, or a dedicated Redis cluster is recommended; they provide two crucial advantages. First, they handle requests before they reach your application servers, meaning that malicious traffic or denial-of-service attempts are stopped before consuming your application resources. Second, these services are distributed and highly available, making them resilient against targeted attacks and highly expensive for attackers to bring down.

With a practical understanding of security controls, we can now apply similar principles to manage resource usage more directly. Let's implement a message quota for authenticated users to control API credit consumption and ensure fair access.

MESSAGE QUOTA IMPLEMENTATION

The message quota is enforced on the backend, specifically within the API route that handles user messages (`/api/chat` in our case). Here's a breakdown of the process:

1. *User identification*—When a user sends a message, the API route identifies the user using their Clerk user ID, obtained from the `getAuth` function from `@clerk/nextjs/server`.
2. *Quota check*—The API route then checks the user's message count for the current day in Upstash Redis.
3. *Increment count*—If the user hasn't exceeded their quota, the message count is incremented in Redis.
4. *Quota exceeded*—If the user has exceeded their quota, an error is returned, preventing the message from being processed.

Figure 9.3 illustrates this simple logic for checking and incrementing the message quota for a user.

How should we implement this? Ideally, we want to place it in our middleware section, but we can also add this check directly to the API handler. The following listing shows how to do it.

Listing 9.6 Integrating message quota limits

```
import { getAuth } from '@clerk/nextjs/server';
const checkMessageQuota = async (userId) => {
  const now = new Date();
  const today = now.toISOString().split('T')[0];
  const key = `message_count:${userId}:${today}`;
```

Imports getAuth from Clerk to retrieve user authentication details

Constructs a Redis key to track daily message counts for a specific user